

DLNA Gateway — Architecture (2.1 · ASGI/Hypercorn)

Four journeys, each read left to right · node codes (P/T/D/E/J) index into the lists overleaf

JOURNEY 1

A track reaches your ears

— what happens when you press play

Files on disk

Music, audiobooks and video on external volumes, read-only.

/volumes/SAMDATA (D/3)

▶

Index

Tags read once per file; a folder is an album.

P/g10 localsfs · P/g9 indexer

▶

library.db

One SQLite index. WAL, FTS5 search, per-thread conns.

P/g7 LibraryDB · P/g8 pool

▶

Choose

Browse or search, then queue to a renderer or the browser.

P/g20 api_browse · P/g15 player

▶

Bytes move

The renderer pulls from :8200 ITSELF — the gateway is not in the audio path. Browser audio uses the Range proxy.

P/g11 localsfs_server · P/g16 /stream

JOURNEY 2

The library learns what it knows

— batch tools you run; never during playback

Tag in place

beets writes clean tags and MBIDs into the files.

T/a13 beets_enrich · T/a14 post_beets

▶

Resolve artists

Artist to MusicBrainz id. THE KEYSTONE: four sources key off it. 2,889/3,289.

T/a15 artist_mbidi

▶

Fetch facts

Life-span, birthplace, genres, biography, best-known, and band line-ups with dates.

T/a16 artist_meta

▶

Fetch credits

Composer/lyricist by browsing each artist's WORKS — 100 per request, an hour not sixteen.

T/a17 track_credits

▶

Housekeeping

Orphan relinks, duplicate and corrupt-file audits, playlist repair. All dry-run by default.

T/a2-a12

JOURNEY 3

A phone clip becomes a browsable memory

— the only journey with machine learning in it

iPhone to Immich

Phone backup. Immich runs FACE RECOGNITION in its own Postgres — never written into the files.

Immich (separate service)

▶

Import originals

Copies Immich's originals, never its transcodes. Dedup by BLAKE2b content hash.

T/a18 immich_import

▶

Scan and title

Every 5 min. Titles from METADATA, not filenames: GPS reverse-geocoded plus capture time. No GPS? inferred from temporal neighbours.

P/g31 video_index · P/g33 geocode

▶

Pull the ML back

Person tags over Immich's REST API, matched by SHA1 CONTENT CHECKSUM — Immich's paths are container paths and cannot be mapped.

T/a19 immich_people_sync

▶

Browse it

By date, country/place, person, or all. Native on the TV; the PWA gets an on-demand HLS transcode.

P/g23 upnp_browse_video · P/g32 ffmpeg

JOURNEY 4

Four ways to reach the same library

— one index, four protocols

PWA (P/c1)

Browse, playback, lyrics, the artist panel, video.

HTTPS/2 :8443 · /api/*

▶

Naim (D/1)

Browses the gateway AS a DLNA Media Server; pulls bytes itself.

UPnP SOAP :8765 /gw/*

▶

CarPlay (P/c2)

Amperfy over Tailscale, including audiobook bookmarks.

Subsonic /rest/*

▶

LG TV (D/5)

The GWMovies tree; plays HEVC and MKV natively.

UPnP · no transcode

▶

Renderers, generally

Any UPnP MediaRenderer. Volume via RenderingControl — hardware, so the audio stays bit-perfect.

P/g14 avtransport

EXTERNAL SERVICES

LIVE = can affect playback · BATCH = only while a tool you started is running

musicbrainz.org

[LIVE] Release-group for album art; artist facts and works for the sweeps. 1 req/s · UA required

*.api.radio-browser.info

[LIVE] Internet-radio station catalogue. DNS round-robin · HLS filtered

en.wikipedia.org

[BATCH] Artist biographies. CC BY-SA: stored only WITH its url. no key · 74% coverage

api.acoustid.org

[BATCH] Fingerprints, reached ONLY by beets in its own process. beets' own key

coverartarchive.org

[LIVE] Front-cover presence for an album MBID. follows 307 to archive.org

nominatim.openstreetmap.org

[LIVE] GPS to place name for video titles. THE PRIVACY-RELEVANT ONE. 1.1 s/req · sticky cache · opt-out

ws.audioscrobbler.com

[BATCH] Last.fm best-known-for, ranked by real listening. APPLICATION key · 100%

lrc.lib.net

[LIVE] Lyrics, on the button. Cached forever, hit or miss. once per track URL

upload.wikimedia.org

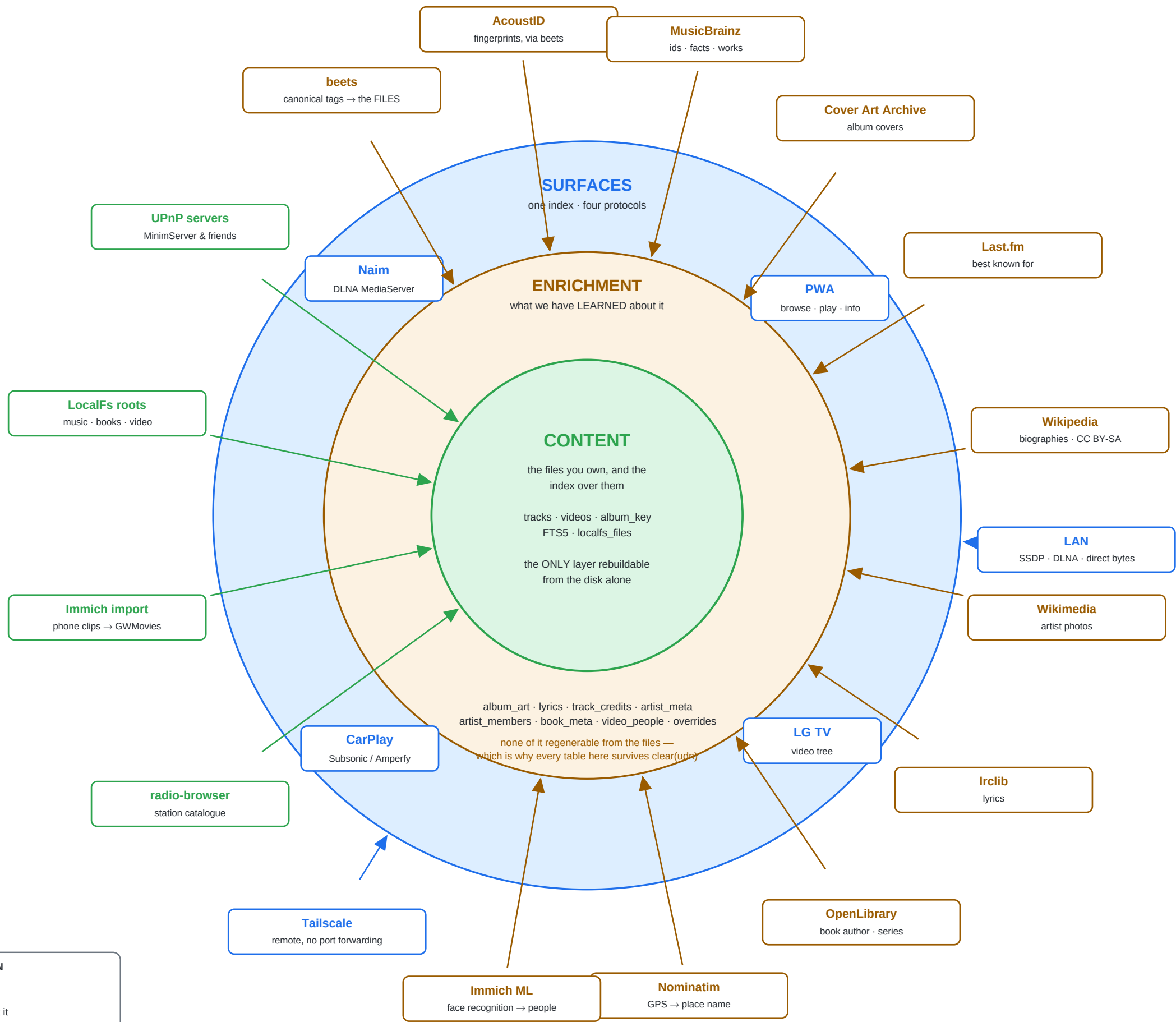
[LIVE] Artist photos, through the /art proxy and its disk cache. freely licensed · cached

openlibrary.org

[BATCH] Audiobook author, title and series number. ~1 req/s · EN/NL only

DLNA Gateway — What feeds what

The same system by STRUCTURE rather than sequence · arrows point at the layer each service actually touches



Most of the outside world feeds ENRICHMENT, not content: the library is yours, the facts about it are borrowed.

Node lists — every box in the drawing

Codes match the diagram. **P** = program / module, **T** = tool, **D** = device, **E** = external service, **J** = scheduled job.

Application programs & modules (P/*)

Code	File / location	Responsibility
P/c1	static/index.html, app.js, app.css, sw.js, manifest.json	PWA front-end. Browse/playback UI, MediaSession lock-screen, Service-Worker caches (APP shell network-first since 2026-06-27 — cache is the offline fallback only, ART cache-first, API stale-while-revalidate), offline shell. Videos section with a same-origin <video> player + vendored hls.js for the on-demand HLS transcode.
P/c2	(3rd-party iOS app — Amperfy / substreamer / play:Sub)	Subsonic client for CarPlay over Tailscale. Talks /rest/* to the gateway; never to the music server directly.
P/g1	dlna_gateway.py	Module-wiring + start_background_services(): spawns the daemon threads (SSDP listener, pre-prober, subnet scanner, heartbeat, gateway announcer). Called from the dlna_asgi lifespan so `hypercorn dlna_asgi:app` boots the whole gateway. Its own stdlib HTTP edge + TLS were removed in 2.0 (Hypercorn owns the edge).
P/g3	dlna_routes.py	GET_ROUTES / POST_ROUTES path→handler maps; the ASGI bridge (P/g28) mounts the not-yet-native ones.
P/g4	dlna_discovery.py	SSDP multicast listener, probe, subnet scanner, renderer/server heartbeat. Holds SERVERS / RENDERERS singletons.
P/g5	dlna_registry.py	Data classes + thread-safe ServerRegistry / RendererRegistry.
P/g6	dlna_devices.py	DeviceRoleCache — in-memory mirror of device_roles for zero-latency classification.
P/g7	dlna_library.py	LibraryDB — SQLite index, FTS5 search, playlists, album_art, play_counts, lyrics, metadata_overrides, favourites, radio, videos + geocode_cache. Composition root for DB-owning singletons + fetchers.
P/g8	db_pool.py	SQLite connection pool — WAL, thread-local conns, write serialization.
P/g9	dlna_indexer.py	Background crawler that walks a provider and populates LibraryDB (with FTS self-heal).
P/g10	dlna_providers/ (__init__, upnp.py, localfs.py, mock.py)	LibraryProvider seam + registry. UpnPProvider wraps the UPnP SOAP path (kept for MinimServer); LocalFsProvider is the in-process backend (mutagen, content-hashed ids, watchdog).
P/g11	dlna_localfs_server.py	RoHaLocalFS bit-perfect file server (:8200). Range-aware (206/416), DLNA-headered, /localfs/stream/ + /localfs/art/ + /localfs/video/ (the LG plays videos from here). Path-traversal defended.
P/g12	dlna_localfs_wiring.py	Boot-time wiring of the LocalFs provider (maybe_start_localfs); gated on LOCALFS_MUSIC_ROOT / config. Also starts the GWMovies video scan (P/g31, VIDEO_UDN) when LOCALFS_VIDEO_ROOT / localfs.video_root is set — incremental, re-run every 5 min.
P/g13	dlna_content.py	UPnP ContentDirectory SOAP client (cd_browse/cd_search). Reached only via P/g10 upnp.py.
P/g14	dlna_avtransport.py	UPnP AVTransport + RenderingControl SOAP (SetURI/Play/Stop/Pause, state/position probe, SetVolume).
P/g15	dlna_player.py	RendererQueue (sequential per-renderer playback, gapless via SetNextAVTransportURI, watchdog) + QueueRegistry (QUEUES, one per UDN).
P/g16	dlna_stream_proxy.py	Browser-audio HTTP proxy /stream (Range pass-through, 5-min idle timeout) + proxy_radio_stream /radio_stream (ICY de-interleave).
P/g17	dlna_art_fetcher.py	AlbumArtFetcher — MusicBrainz release-group THEN release + Cover Art Archive lookup; sticky notfound cache; ~1 req/s. Art attaches to a RELEASE, so a release-group reports none unless a pressing is flagged as its cover — which is why compilations need the fallback. bare_albums() takes three opt-in filters (skip audiobooks, skip 1-2 track strays, report the performer count) because a rate-limited budget spent on impossible questions is the whole cost.
P/g19	dlna_lyrics.py	On-demand IrcLib lyrics; cached in the lyrics table; sticky positive + negative.
P/g20	api_browse.py	Browse / search / radio-shuffle API.
P/g21	api_playback.py	Playback, /stream route, /art proxy, /api/client_log, state, indexer management.
P/g22	api_playlists.py	Playlist CRUD endpoints.
P/g23	api_upnp.py	The gateway-as-MediaServer: a COMPLETE DLNA Media Server the Naim/LG browse. device.xml (MediaServer + X_DLNA DOC + icons + ContentDirectory AND ConnectionManager), both SCPDs, ContentDirectory#Browse over the full library + the pre-browse handshake actions, ConnectionManager #GetProtocolInfo, GENA SUBSCRIBE + initial NOTIFY, SSDP announce + M-SEARCH responder. Root also lists a Videos folder (only when videos exist) so the LG TV browses + plays GWMovies natively (HEVC/MKV incl.).
P/g24	api_subsonic.py	Subsonic-compatible /rest/* API (browse, playlists, favourites→starred, stream, cover, scrobble, internet radio) for CarPlay clients.
P/g25	api_radio.py	Internet-radio endpoints: radio-browser search, favourites (≤25), ICY now-playing.
P/g26	dlna_config.py	Constants (DB_FILE/CFG_FILE/LOG_FILE), logging setup, config load/save, .env load (if dotenv present), raise_fd_limit(8192).
P/g27	dlna_asgi.py	2.0 — THE server (Hypercorn owns the whole edge). FastAPI app: TLS+HTTP/2 on :8443 via ALPN, plain :8765; owns the tailscale cert. Native routes for the read API, /art, /stream + /radio_stream relays, static/PWA, the Subsonic byte methods, the video surface (/api/videos, /video/<id>, /video_poster, /video_transcode, /video_hls — same-origin so the PWA's <video> works over HTTPS), and the Naim/LG-facing /gw/* UPnP surface (device.xml/desc.xml/events/control on the plain :8765 bind — Cleanup C folded it in here, retiring the separate device server). Remaining legacy handlers run via the bridge. Lifespan boots P/g1 services. docs_url off (no CDN call).
P/g28	dlna_asgi_bridge.py	Shim that runs the legacy (h, params) handlers unchanged inside the ASGI app (a fake `h` captures _json/_html/_xml/send_error; runs in a threadpool). Routes are rewritten native one batch at a time, then dropped from the bridge.
P/g29	dlna_art_cache.py	On-disk cover-art byte cache keyed by source URL. art_fetch_cached() (in P/g21) fronts art_fetch so /art and Subsonic getCoverArt serve repeat covers from disk (across clients + restarts) instead of re-fetching coverartarchive / re-decoding embedded art. TTL + size capped; art_cache/ gitignored.
P/g30	dlna_events.py	EventBus/EVENTS — thread-safe publish to the asyncio loop; native GET /api/events (SSE). Publishers: RendererQueue state, index status transitions, discovery changes. The PWA opens an EventSource as a polling accelerator (fallback intact).
P/g31	dlna_video_index.py	GWMovies scanner (V1) → the videos table. Stable id = sha1(rel_path); metadata via P/g32 probe (filename/mtime fallback when ffprobe absent); place name via P/g33; poster frame extracted per video. Incremental (mtime,size) + prunes gone files; force=True rebuilds.
P/g32	dlna_ffmpeg.py	Optional ffmpeg/ffprobe helpers (V0/V3, same posture as fpcalc — absent binaries degrade gracefully). probe() metadata/GPS/capture time, extract_poster(), transcode_cmd() for the on-demand H.264/AAC HLS transcode (-pix_fmt yuv420p so 10-bit HEVC plays), build_display_title() for the '<place>_YYYYMMDD_HHMM' fallback title.
P/g33	dlna_geocode.py	Reverse-geocode GPS → place name via Nominatim/OSM (E/x6), cache-first in geocode_cache. UA + 1.1 s rate limit per OSM policy; failures NOT cached (retry later), definitive no-name cached sticky as ". PRIVACY: this sends your clips' GPS coordinates off-machine automatically; opt out by not setting LOCALFS_VIDEO_ROOT.
P/g34	dlna_ssrf.py	SECURITY (2.1) — the outbound-fetch guard on every caller-supplied ?url= (/art, /stream, /radio_stream) and every redirect hop. Refuses private / loopback / link-local destinations unless the host is a device already in SERVERS/RENDERERS (so the LocalFs file server on :8200 stays reachable) and refuses non-http(s) schemes. Before it, /stream relayed ANY internal HTTP service's body verbatim and /art's error text was an open/closed/filtered port oracle. Refusals are logged; the caller gets a uniform failure.
P/g35	dlna_credits.py	Songwriting credits (2026-09-20): is this composer/lyricist a NAME or machine junk? DISPLAY-only — the file tag and the tracks row are untouched. 8% of this library's credits are scene adverts (www.t.me/...). REJECTED rule, kept as a test: "credits with no A-Za-z letter are junk" matched 55 rows, every one a real composer (■■■■■■■■■■■, ■■■■■■■■■■).
P/g36	dlna_mbid.py	The artist → MusicBrainz-id decision, pure. Every external source keys off this id, so one wrong match is wrong in four places and never invites correction — it REFUSES rather than guesses. & is never split (band names contain it), commas never at all, every dash folds to a space (MB spells names with typographic dashes), aliases only when nothing matched by name, and two artists sharing a name are refused.
P/g37	dlna_lineup.py	"Who was in the band when this was recorded?" — pure date-intersection over membership. Exists because per-recording performer credits cover only ~17% of tracks while membership is near-universal. Members REJOIN (Richard Wright has two stints), and no year means no claim.

Code	File / location	Responsibility
P/g38	dlna_artist_fetch.py	The artist-metadata sources — MusicBrainz (identity, life-span, genres, members), Wikipedia (bio, stored only WITH its url: CC BY-SA attribution), Last.fm (best-known, needs an application key). Parsing is split from HTTP so the judgement is testable. Owns the `member of band` DIRECTION rule: backward = a group's members, forward = a person's bands.
P/g39	dlna_asgi_artist.py	GET /api/artist_info — the artist panel's ONE request. Resolves the line-up for the track's year and LABELS it (inferred vs credits) so the client cannot promote a guess to a fact. Own module: dlna_asgi_browse is at 397/400 lines.
P/g40	dlna_library_artists.py	ArtistsMixin — artist_meta (id + display facts) and artist_members (line-up). Survives clear(udn): each row cost a rate-limited round-trip. Keyed by the NORMALISED name so spelling variants are one question, not three.
P/g41	dlna_art_query.py	The PURE half of a cover lookup: one (artist, album) → an ORDERED list of attempts. The exact pair is ALWAYS first so nothing that resolves today can regress; looser forms follow in increasing order of risk, and title-only is offered only for multi-artist folders where the artist is guaranteed to fail. tidy_album is LOOKUP ONLY — a loose query wastes one request, a loose identity merges two albums.
P/g42	dlna_providers/localfs_art.py	What an album LOOKS like: the embedded picture, and the cover file sitting BESIDE the music when there is none. Choosing which image is the whole problem — “biggest wins” picks back.jpg, “first wins” picks a 75px WMP thumbnail, so selection is name-first with a size floor only as a last resort, and rejections match whole WORDS (“cd” would eat “ACDC - cover.jpg”). A disc subfolder also looks in its parent; an ordinary folder never climbs.
P/g43	dlna_library_credits.py	CreditsMixin — track_credits, composer/lyricist browsed off MusicBrainz works for the ~68% of tracks whose files carry none. Not tracks.composer, because clear(udn) would throw away a night's fetching. THE FILE TAG WINS on read.
P/g44	dlna_library_search.py	SearchMixin — the FTSS free-text question, split from browse at exactly 400 lines. Navigating a hierarchy and answering a question are different jobs with different semantics.

Maintenance tools (T/*) — tools/*.py

Operate directly on library.db and/or the music root. All default to dry-run / report-only; deletions move to Trash unless --hard-delete. Each has a unit-test sibling tools/test_*.py.

Code	File	Purpose & key options
T/a1	regen_schema.py	Regenerate the committed schema.sql artifact after any schema change. --check = fail if stale (the test_schema_sync gate).
T/a2	prune_empty_music_dirs.py	Trash directories whose subtree has zero music files. --dry-run -v --limit N --exts a,b --hard-delete -y.
T/a3	find_corrupt_audio.py	Flag files with bad/zero magic bytes for their extension. Writes corrupt-audio.txt. --trash --hard-delete --limit --exts --out -y.
T/a4	find_duplicate_audio.py	Find duplicate recordings on disk (same acoustid metadata); ranks a winner by bit-depth/sample-rate/size. --trash --hard-delete -v -y.
T/a6	relink_orphan_overrides.py	Relink metadata_overrides orphaned by a UPnP rescan via d-id + fuzzy (artist,title). --apply --db.
T/a7	relink_playlists_to_locals.py	Repoint playlist_tracks/favourites at RoHaLocalFS by normalised metadata; prune no-match. --apply --no-backup --no-prune-favs -y.
T/a8	audit_override_mismatches.py	Delete acoustid overrides whose artist AND title both mismatch the track (d-id collision repair). --clean --top -y.
T/a9	correct_year_drift.py	Rewrite metadata_overrides.year to the earliest plausible year evidenced by another copy in the library. --apply --top --db -y.
T/a10	improve_song_years.py	Query MusicBrainz for each song's earliest recording year → song_year_cache → apply. --lookup --apply --limit -v.
T/a11	locals_scan.py	Standalone LocalFs indexer (CLI). NOTE: do not run against the live DB without a base_url — leaves placeholder URLs.
T/a12	locals_serve.py	Standalone LocalFs file-server launcher for testing the :8200 serving path.
T/a13	beets_enrich.py	Run the beets tag-in-place enrichment batch (docs/enrichment.md). Safe wrapper around `beet import`: enforces write:yes/copy:no/move:no before any write. --write-config --quiet --timid --album --revisit --reindex --gateway --dry-run -y.
T/a14	post_beets_reindex.py	The post-beets step: drop the historical AcoustID metadata_overrides (LocalFs URLs are path-stable, so old acoustid rows would re-mask beets' fresh tags) then reindex LocalFs. --apply --dry-run --no-clean --no-reindex --no-backup --udn --gateway --db -y.
T/a15	artist_mbid.py	Resolve every artist to a MusicBrainz id — the keystone the rest of the artist metadata hangs off. Tags first (free, exact), then name search. Live: 2,889 artists, 87.8% resolved, 85.9% search acceptance. Resumable (sticky notfound). --apply --limit N --tags-only -v.
T/a16	artist_meta.py	Turn each id into the panel's facts: life-span, birthplace, genres, biography, best-known, and a band's members with instruments + dates. Live: 2,889 in 3h37m, 0 errors. SET LASTFM_API_KEY FIRST — meta_fetched_at marks an artist done, so a keyless pass leaves them all without best-known. --apply --limit N -v.

LAN devices (D/*), external services (E/*), scheduled jobs (J/*)

Code	Name	Role / protocol
D/1	Naim Uniti	UPnP MediaRenderer. Receives AVTransport+RenderingControl SOAP (control/volume); pulls audio bytes over HTTP Range from :8200; also browses gateway playlists as a control point. Gateway is NOT in its audio path → bit-perfect.
D/2	UPnP MediaServer	MinimServer / generic UPnP (AssetUPnP decommissioned). Browsed via P/g10 UpnpProvider + P/g13 ContentDirectory SOAP. Kept for any non-LocalFs source.
D/3	Media files	/Volumes/SAMDATA/Music + /Volumes/SAMDATA/GWMovies (external drive, read-only). Audio: indexed by P/g9, tags read by locals provider, bytes served by P/g11. Video: scanned by P/g31, served by P/g11 (/locals/video).
D/4	fpcalc · ffmpeg/ffprobe	Local CLI binaries (brew). fpcalc fingerprints for the beets tool (T/a13); ffmpeg/ffprobe probe metadata, extract posters and run the on-demand HLS transcode (P/g32). Internal/local — not network.
D/5	LG TV (webOS)	DLNA control point + player. Browses the gateway's /gw MediaServer (incl. the Videos folder) and pulls bytes from :8200. Plays HEVC/MKV natively — the PWA transcode path is browser-only.
E/x1	musicbrainz.org	GET /ws/2/release-group — MBID + original year. UA + 1.1 s rate limit required.
E/x2	coverartarchive.org	HEAD /release-group/{mbid}/front-500 — cover presence.
E/x3	Irclib.net	GET /api/get — on-demand lyrics for the playing track.
E/x4	*.api.radio-browser.info	GET /json/stations/search — internet-radio catalogue (HLS filtered).
E/x5	api.acoustid.org	fingerprint → MusicBrainz metadata — used ONLY by the beets tool (T/a13) now; the in-process AcoustID worker was removed in 2.0.
E/x6	nominatim.openstreetmap.org	GET /reverse — GPS → place name for video display titles (P/g33). UA + 1.1 s rate limit per OSM policy; sticky cache in geocode_cache.
E/x7	en.wikipedia.org	GET /api/rest_v1/page/summary/{title} — artist biography for the artist panel. CC BY-SA: the bio is DROPPED if its url is missing, because the link is the attribution. Disambiguation stubs refused.
E/x8	ws.audioscrobbler.com	Last.fm artist.getTopTracks — “best known for”, ranked by real listening. Needs an APPLICATION key (LASTFM_API_KEY); an account password is NOT an API credential. Absent → the block is omitted.
J/1	com.roha.dlna-gateway	LaunchAgent that runs the gateway. Restart: launchctl kickstart -k gui/\${id -u}/com.roha.dlna-gateway.
J/2	com.roha.dlna-cert-renew + renew-cert.sh	Weekly TLS cert renewal (Mon 04:30; no-op unless <30 days). cert-renewal.log.
J/4	setup.sh	venv setup + run / restart / probe. --run --restart --no-browser --debug --probe URL --list-devices --reset-devices. See the setup.sh options reference.

Commands & options used in the drawing

Context	Command
Run (J/4)	./setup.sh --run [--no-browser] [--debug] [--probe http://...] [--list-devices] [--reset-devices]
Restart (J/4)	./setup.sh --restart (refresh venv/deps + launchctl kickstart -k gui/(id -u)/com.roha.dlna-gateway)
Cert (J/2)	./renew-cert.sh [--force] · launchctl kickstart gui/(id -u)/com.roha.dlna-cert-renew
Subsonic auth	launchctl setenv SUBSONIC_PASSWORD ... ; launchctl getenv SUBSONIC_PASSWORD
Full test suite	python tests/run_all.py [--offline --frontend --frontend-only http://host:8765]
Unit / Playwright	python3 -m unittest tests.test_player ... ; .venv/bin/pytest tests/frontend -v
Chaos (live)	python3 tests/chaos.py --iterations N --workers M [--seed S] [--quiet] [--base https://host:8443]
Load (live)	python3 tests/load_stream.py --concurrency 40 --count 80 [--gateway https://127.0.0.1:8443 --insecure] [--max-p95 6] (threadpool-starvation guard)
Safari / iOS smoke	.venv/bin/python tests/frontend/safari_smoke.py · ios_sim_smoke.py (Appium :4723 + a booted Simulator) — opt-in, not in run_all.py
Schema gate (T/a1)	python3 tools/regen_schema.py [--check]
Module self-tests	python dlna_discovery.py dlna_content.py dlna_library.py db_pool.py dlna_player.py
Range / 206 check	curl -r 0-1023 -D - http://<host>:8200/locals/stream/<id> -o /dev/null
Tools (T/a2–a12)	python3 tools/<tool>.py [--dry-run --apply --trash --hard-delete --limit N --exts a,b -v -y] (see the tools table above)

Ports (2.0): 8443 HTTPS — Hypercorn TLS + HTTP/2 (ALPN), tailscale cert · 8765 plain HTTP, incl. the Naim/LG-facing /gw/* UPnP surface (Cleanup C folded the old :8770 device tier into this bind; SSDP advert points here) · 8200 RoHaLocalFS file server (audio + /locals/video) · 26125 (legacy AssetUPnP, decommissioned). The HTTP/2 + app-owned-TLS roadmap is now DONE — Hypercorn terminates TLS/h2 natively.

Tool options reference — what each flag does

Every option for every maintenance tool, with its meaning. Codes match the diagram (T/aN) and the tools table. Unless noted, file-deleting tools default to dry-run / report-only and move to the macOS Trash (recoverable ~30 days) — **--hard-delete** is the permanent, non-recoverable variant.

T/a1 regen_schema.py — Regenerate the committed schema.sql.

Option	Meaning
(no args)	Rewrite schema.sql from the live LibraryDB schema.
--check	Don't write; exit non-zero if schema.sql is stale (the test_schema_sync gate).

T/a2 prune_empty_music_dirs.py — Trash directories whose subtree has zero music files.

Option	Meaning
	Music root to walk (positional).
--dry-run	Print decisions; touch nothing.
-v / --verbose	Also log every KEPT directory (default: only deletions print).
--limit N	Stop after evaluating N dirs; when the limit is hit, NO deletions run (a partial picture = safety belt).
--exts a,b,c	Override the music-extension set (commas, dot optional).
--hard-delete	Permanent rm -rf instead of Trash. NOT recoverable.
-y / --yes	Skip the confirmation prompt.

T/a3 find_corrupt_audio.py — Flag files whose magic bytes are wrong/zero for their extension.

Option	Meaning
	Music root to scan (positional).
-v / --verbose	Also log every OK file (default: only corrupt).
--limit N	Stop after scanning N files (0 = no limit); halts the delete step when hit.
--exts a,b,c	Override the audio-extension list.
--out PATH	Where to write the corrupt-paths list (default ./corrupt-audio.txt; pass /dev/null to suppress).
--trash	Move flagged files to the Trash.
--hard-delete	Permanent unlink instead of Trash. Mutually exclusive with --trash.
-y / --yes	Skip the confirmation prompt before deleting.

T/a4 find_duplicate_audio.py — Find duplicate recordings on disk (same acoustid metadata); keeps a ranked winner.

Option	Meaning
	Music root to scan (positional).
-v / --verbose	Per-URL ambiguity / not-found logs.
--trash	Move the loser files to the Trash (winner kept).
--hard-delete	Permanent delete of losers. NOT recoverable.
-y / --yes	Skip the confirmation prompt.

T/a6 relink_orphan_overrides.py — Relink metadata_overrides orphaned by a UPnP rescan (d-id + fuzzy artist/title).

Option	Meaning
(default)	Dry-run preview — no mutation.
--apply	Perform the relinks.
--db PATH	library.db path.

T/a7 relink_playlists_to_locals.py — Repoint playlists/favourites at RoHaLocalFS by normalised metadata.

Option	Meaning
(default)	Dry-run preview.
--apply	Commit the relinks (auto-backs up library.db first).
--no-backup	Skip the automatic library.db backup on --apply.
--no-prune-favs	Keep album_favourites that no longer match any LocalFs album.
-y / --yes	Skip the confirmation prompt.

T/a8 audit_override_mismatches.py — Delete acoustid overrides whose artist AND title both mismatch the track.

Option	Meaning
(default)	Dry-run, lists the top 30 suspects.
--clean	Delete the suspect acoustid override rows.
--top N	Show N rows (0 = full list).
-y / --yes	Non-interactive (skip the confirm).

T/a9 correct_year_drift.py — Rewrite metadata_overrides.year to the earliest plausible year from another copy in the library.

Option	Meaning
(default)	Dry-run, top 30 candidates.
--apply	Write the corrections (as source='manual').
--top N	Preview N candidates (0 = all).
--db PATH	library.db path.
-y / --yes	Non-interactive apply.

T/a10 improve_song_years.py — Query MusicBrainz for each song's earliest recording year → cache → apply.

Option	Meaning
(default)	Dry-run preview — no MB calls, no DB writes.
--lookup	Query MB for uncached (artist,title) groups; fill song_year_cache (sticky).
--apply	Write cached hits onto tracks (metadata_overrides.year, source='manual').
--limit N	Cap the number of groups queried (for testing).
-v / --verbose	Per-group logging.

T/a11 localfs_scan.py — Standalone LocalFs indexer (CLI). Don't run against the live DB without a base_url — it leaves placeholder URLs.

Option	Meaning
--root PATH	Music root to scan (default \$LOCALFS_MUSIC_ROOT or /Volumes/SAMDATA/Music).
--force	Ignore the (mtime, size) cache and re-tag every file. Use after schema changes.
--compare	Skip scanning — just print tracks/albums per UDN currently in library.db.
--db PATH	library.db path.
-v / --verbose	Per-file logging.

T/a12 localfs_serve.py — Standalone RoHaLocalFS file-server launcher (test the :8200 path).

Option	Meaning
--port N	HTTP port (default 8200 / \$LOCALFS_PORT).
--host ADDR	Listen address (default 0.0.0.0 so the Naim can reach it).
--db PATH	library.db path.
--root PATH	Allowed music root; repeatable. Defaults to \$LOCALFS_MUSIC_ROOT.
-v / --verbose	Request logging.

T/a13 beets_enrich.py — Run the beets tag-in-place enrichment batch (docs/enrichment.md). Safe wrapper around `beet import`. Deps: brew install chromaprint beets (NOT pip — a Homebrew python upgrade wipes a pip install; that is how the 2026-06 install died), then add musicbrainzngs + pyacoustid to the keg venv (beets 2.x pluginized MusicBrainz — the musicbrainz plugin + musicbrainzngs are REQUIRED or it matches nothing).

Option	Meaning
(no --quiet/--timid)	Interactive: beets prompts you per album (apply / skip / ...); strong matches auto-apply.
--write-config	Write the prog-tuned tag-in-place ~/.config/beets/config.yaml (enables the musicbrainz metadata plugin; backs up any existing) and exit. Run this first.
--music-root PATH	Library root to import (default /Volumes/SAMDATA/Music).
--album PATH	Import a single album directory instead of the whole root.
--config PATH	beets config path (default ~/.config/beets/config.yaml).
--quiet	Unattended BULK pass: auto-accept only strong matches (≥ strong_rec_thresh 0.80), skip the rest, no prompts (beet import -q).
--timid	Most cautious: prompt for EVERY match (per-change review). Mutually exclusive with --quiet.
--revisit	Re-import a directory beets already recorded as done (-I / noincremental). Use after re-tagging an album.
--reindex	After import, find the LocalFs server UDN via /api/servers and POST /api/index/rebuild so the gateway picks up the new tags.
--gateway URL	Gateway base URL for --reindex (default http://127.0.0.1:8765).
--udn UDN	Explicit server UDN to reindex (default: auto-pick the uuid:localfs-* server).
-n / --dry-run	Print the resolved beet command + safety report; do NOT invoke beets (beets has no true dry-run of its own).
-y / --yes	Skip the in-place-write backup-warning confirmation.

T/a14 post_beets_reindex.py — After beets has tagged files in place, make its work visible: clear the AcoustID metadata_overrides (LocalFs URLs are PATH-stable, so the COALESCE pass would otherwise re-lay old acoustid rows over beets' fresh tags) THEN reindex LocalFs. source='manual' is never touched; notfound/video_skip carry NULL metadata so they mask nothing.

Option	Meaning
(default)	Dry-run: print the override breakdown + the planned clear/reindex; change nothing.
--apply	Actually delete acoustid overrides + start the reindex (auto-backs up library.db first).
-n / --dry-run	Explicit preview alias (the default when --apply is absent).
--no-clean	Skip clearing overrides (reindex only).
--no-reindex	Skip the reindex (clean only).
--no-backup	Skip the library.db backup before deleting.
--udn UDN	Server UDN to reindex (default: auto-pick uuid:localfs-*).
--gateway URL	Gateway base URL (default http://127.0.0.1:8765).
--db PATH	library.db path.
-y / --yes	Skip the confirmation prompt.

J/4 setup.sh — Bootstrap + run the gateway. Finds Python 3.14+, creates/repairs the .venv, installs requirements.txt, then either runs the gateway (--run, exec in the foreground), restarts the launchd-managed copy (--restart), or just finishes setup. Unknown flags after --run are forwarded verbatim to dlina_gateway.py.

Option	Meaning
(no flags)	Set up only: venv + dependencies, then print the 'how to run' summary. Does not start anything.
--run	Set up, then exec the gateway in the foreground on :8765 (blocks; Ctrl-C to stop). All flags below it are passed through to dlina_gateway.py.
--restart	Refresh the venv/deps, then restart the launchd gateway via launchctl kickstart -k gui/\$(id -u)/com.roha.dlina-gateway (the launchd-correct restart — a bare kill races launchd's respawn). Aborts with install hints if the LaunchAgent isn't loaded. Mutually informative with --run; --restart wins if both are given.
--no-browser	(forwarded) Don't auto-open the browser on start.
--debug	(forwarded) Verbose logging.
--probe http://...	(forwarded) Add a UPnP server manually by its device-description URL.
--list-devices	(forwarded) Print the known-devices table and exit.
--reset-devices	(forwarded) Wipe the device DB and exit.
--port N	(forwarded) HTTP port (default 8765).